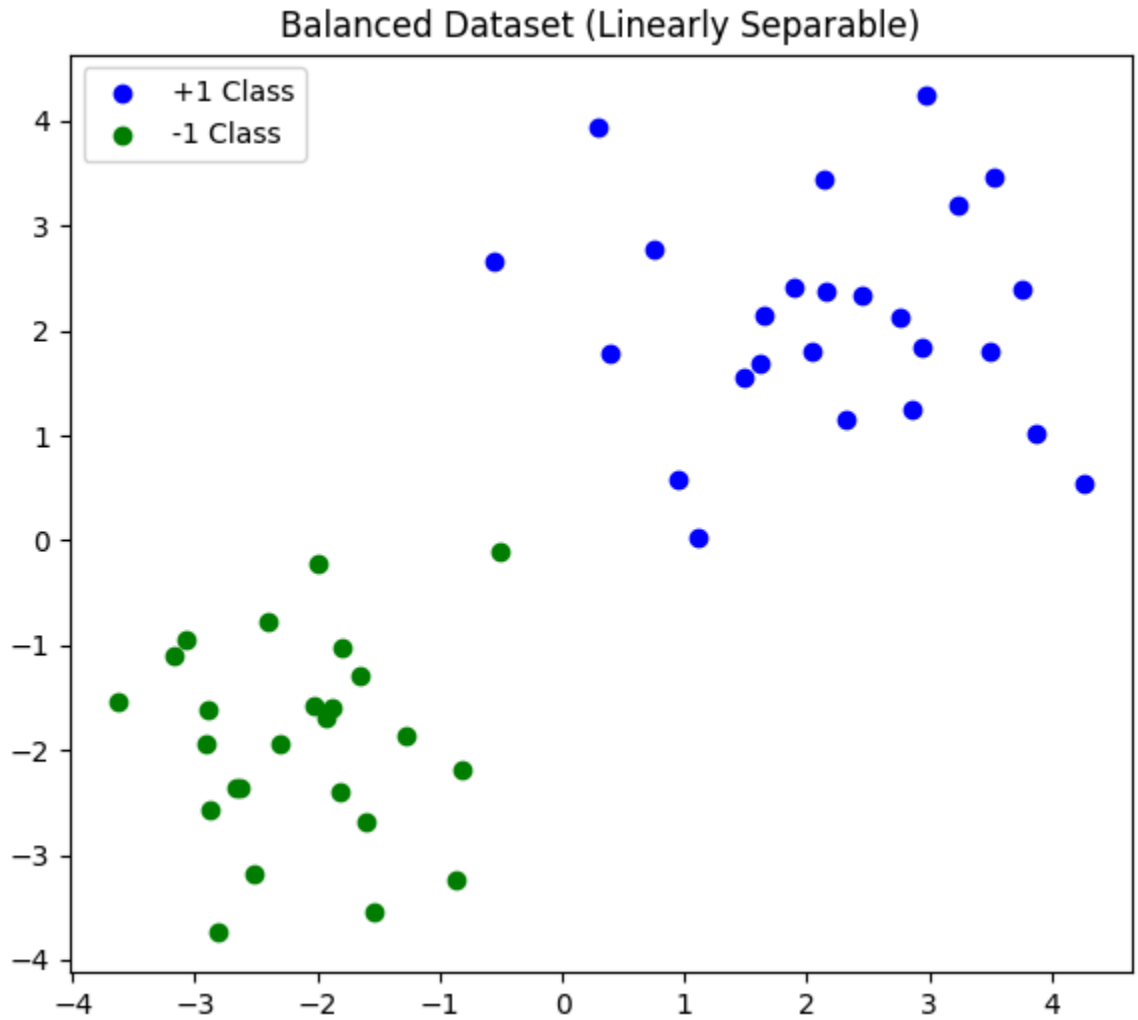
Simon Edmunds
12590227

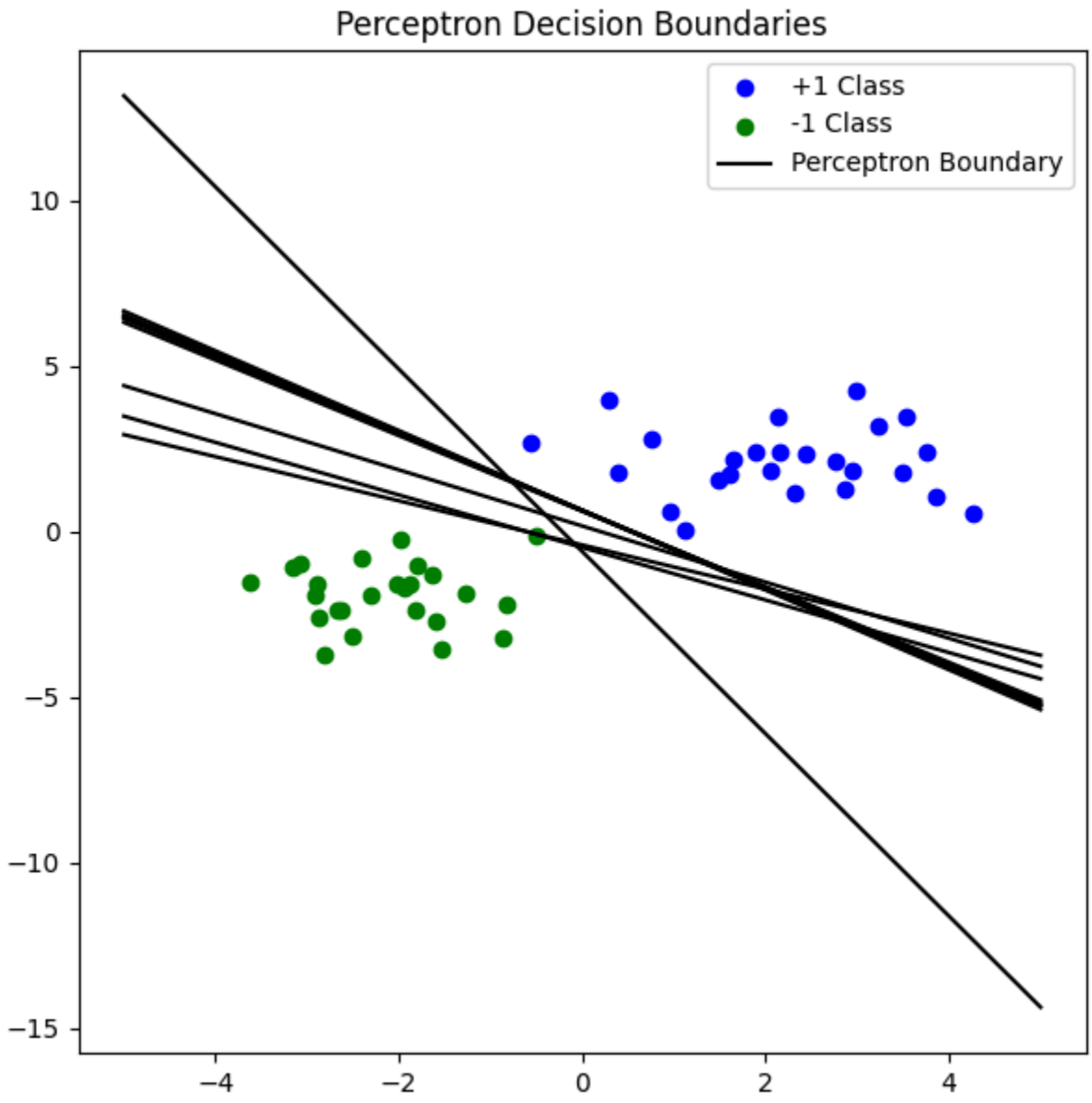
**1. <u>Constructed Dataset</u>**

I created a linearly separable dataset of 50 samples, randomly generating using NumPy:



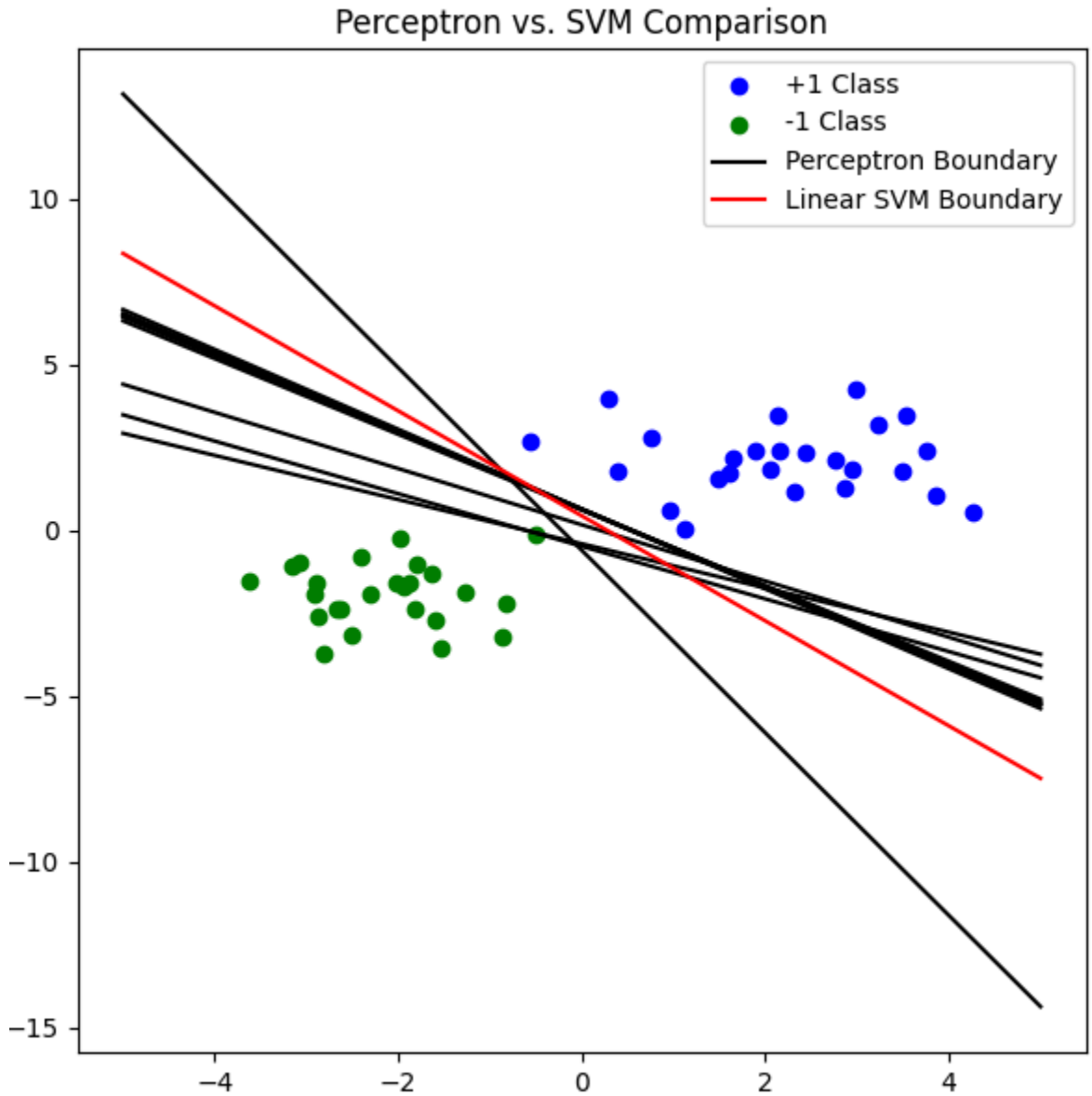
**2. <u>Perceptron Experiments</u>**

I used my perceptron implementation from Task 2 with 10 random initializations for W and b. All of the models were able to separate the classes, but some were definitely better than others:



**3. <u>SVM Comparison</u>**

I used scikit-learn to train a linear SVM model, and compared it to my perceptron models:

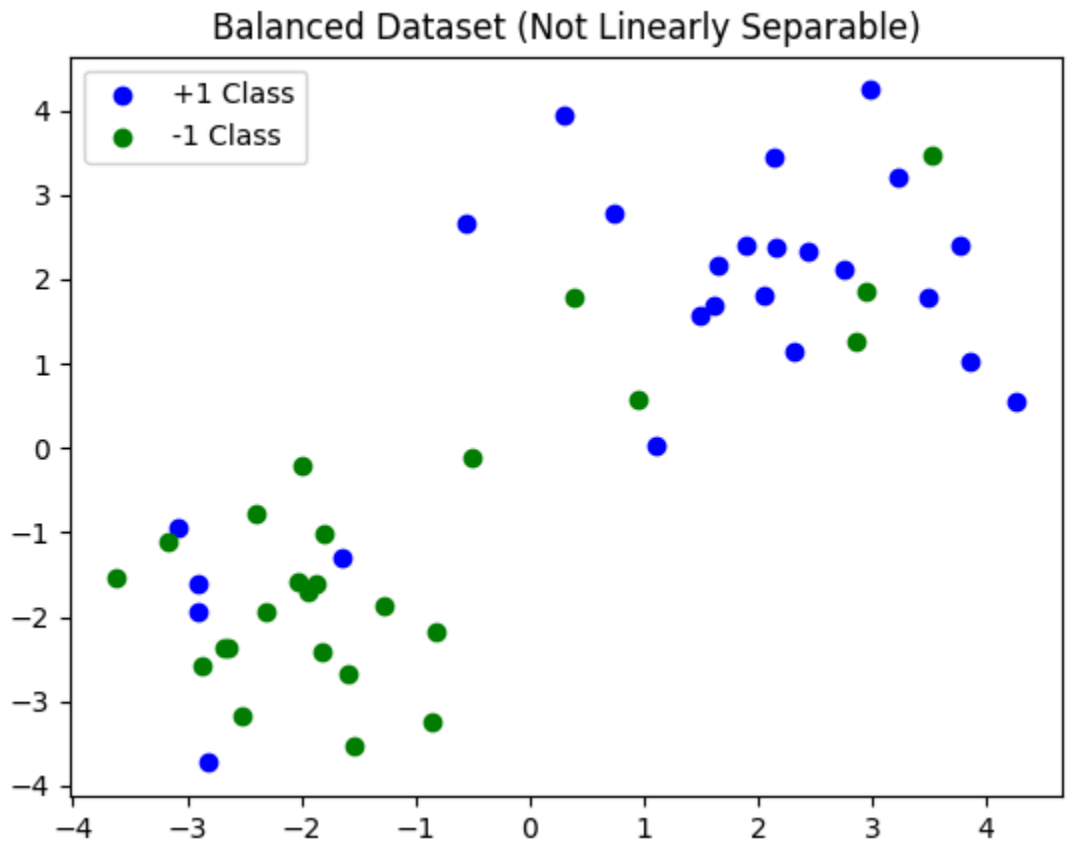



**4. <u>Analysis</u>**

Of the 11 hyperplanes, the SVM hyperplane is definitely the best. Mathematically, SVM is specifically maximizing the geometric margin, so the final hyperplane is as far as possible from the closest training points. On the other hand, perceptrons only try to ensure that the data is separated, meaning that there is no consistent “solution”, and that perceptrons are inherently less stable. Because of this, perceptrons are much more sensitive to initialization, as shown in the plots above (even though the perceptrons separate the data, they have wildly different slopes and offsets geometrically).

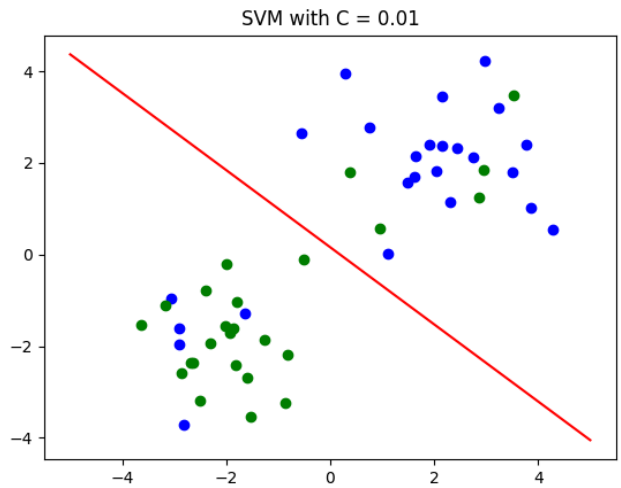
**5. <u>Add Label Noise</u>**

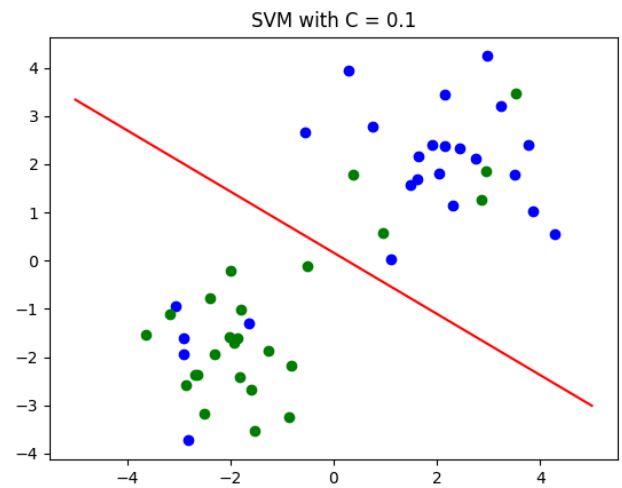
Using NumPy, I randomly selected and flipped 5 data points from each class, making the dataset no longer linearly separable:

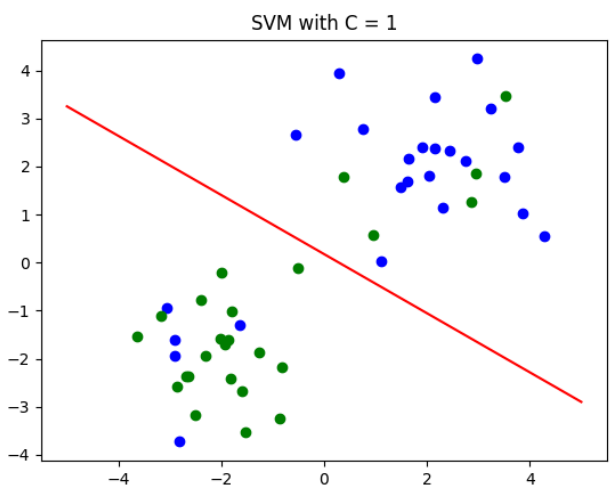


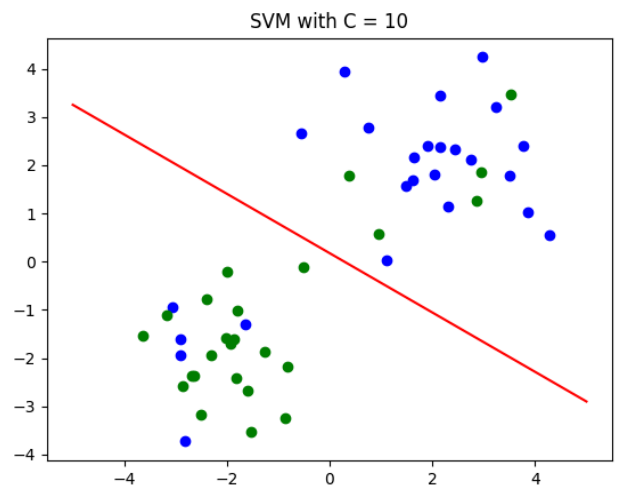
## 6. Soft Margin SVM Study

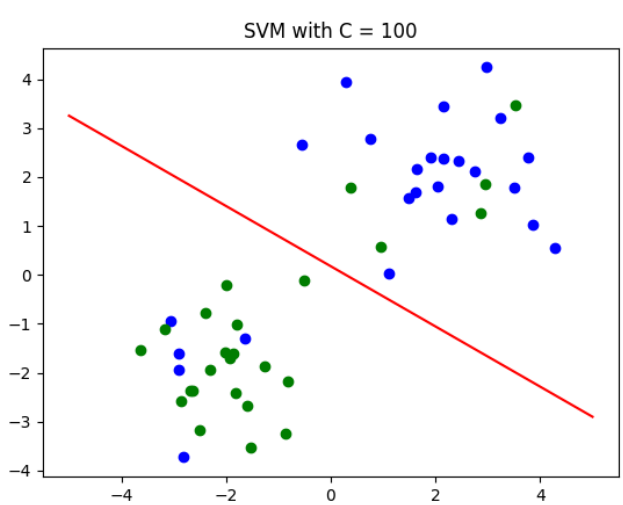
Using the provided list of C values to train five different SVM models on the noisy data, I got the following boundaries:



Explanation:

- When C is very small, the SVM prefers maximizing the margin over minimizing errors. This gives a wider margin and more tolerance for misclassified points (but adds the risk of underfitting).
- When C is very large, the model heavily penalizes errors, causing a narrower margin and slightly pivoting the decision boundary closer to the error points (trying to classify all points correctly).
- In this experiment, because only a small number of points were flipped, overfitting was very minimal with increasing values of C. The decision boundary “overfit” very slightly, as it rotated to try and match the flipped points. This caused a smaller margin overall, and decreased the effectiveness of the model.
- Overall, C controls the tradeoff between margin maximization and training error: small C aims for a large margin, while large C tries to minimize classification errors (smaller margin).